IP Reuse Creation for System-on-a-Chip Design

Pierre J. Bricaud

Mentor Graphics Corporation
Sophia Antipolis, France

Abstract

The never ending increase of silicon capacity available to system and IC designers, as predicted by Moore's Law, brings on a cyclical crisis in design methodology and engineering productivity generating a ripple effect through the EDA and electronics industries. The System-on-a-Chip era will need more than available silicon to become a reality. A new design methodology roadmap based on IP reuse needs to emerge. Two of the EDA giants, Synopsys and Mentor Graphics, took the initiative at DAC 1997 to set the pace for the new challenge of System-on-a-Chip design. After more than a year and the publishing of the Reuse Methodology Manual (RMM) that sets the stage for IP Reuse and System-on-a-Chip design, where do we stand? The Reuse Methodology Manual is well perceived and accepted by the design community and represents a stake in the ground towards ensuring rapid creation of reusable designs. Throughout this tutorial an attempt is made to describe the total SoC design flow based on reusable IP and will also outline some non-trivial issues during this process: effect of available silicon capacity, SoC integration, SoC verification and documentation.

Introduction

Throughout this paper the words 'core', 'macro' or 'block' will be used interchangeably. Same holds for macro 'creator' or 'author'. System-on-a-Chip is the challenge given to the design community by the semiconductor industry for the next millennium. The silicon is available, the new product specifications are being written but the design methodology, EDA tools and necessary building blocks are still in infancy. The first milestone of the SoC design roadmap has been set with the publishing of the Reuse Methodology Manual for System-on-a-chip Designs (RMM) (1). Throughout this tutorial we will focus on methodology of the design flows. The common sense rules and guidelines to RTL coding and synthesis scripting are tutorials by themselves and will not be discussed. The overall design process of the IP reuse creation for SoC design will be separated into system-level specification, soft-hard macro creation and productization, SoC integration and SoC verification. Some thoughts are also given on how to solve the documentation crisis.

IC Design Cycle

Let's choose a simple IC creation cycle: Idea - Design - Verify - Implement. For the Design - Verify, some industry wide accepted ratios for each specific task are:

Table 1. Industry Ratios

Task	Time Ratio (%)
Verification	40
RTL + Synthesis	20
IC Layout	10
Test	10
System Integration	10
Other	10

Source: EDA Today

These ratios outline where the research effort must be directed to, immediately: verification. Facing the increasing complexity of the verification process, the natural solution is to adopt a "Divide and Conquer" strategy. You start the verification process in a hierarchical manner with what we can call the SoC leaf cells: reusable IP cores. This is your first necessary investment towards success. The other tasks of the design cycle will stay relatively the same,

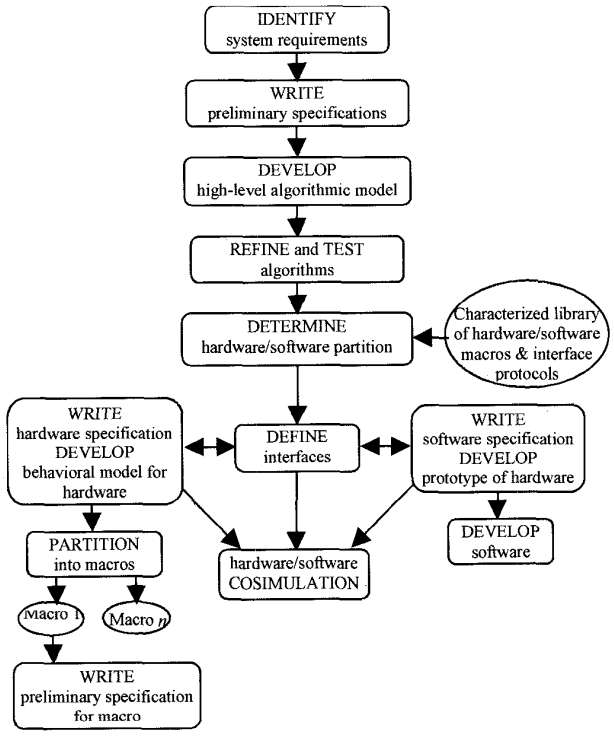


Figure 1. The System-on-a-Chip Design Process

0-7803-5443-5/99/$10.00©1999 IEEE

Authorized licensed use limited to: ETRI. Downloaded on October 14,2026 at 02:31:45 UTC from IEEE Xplore. Restrictions apply.

except for system integration. If the latter is not well specified at the beginning, especially at the hardware/software level, as you can see in fig. 1 (Fig. 2-4, RMM), the multi-generation-engineer SoC design is born and system integration will become a nightmare. On the positive side, your reusable IP creation investment will give you the solution, almost free of charge, for what we can call the 'Silicon White Space Hole' (2).

The Silicon White Space Hole is the available silicon left that a system designer has to fill after using his created IPs and memory blocks. For example, the amount of 'equivalent' silicon gates available or memory elements available is orders of magnitude above what can be designed from scratch in a reasonable time frame. Just to give some numbers that we can visualize: An HDL designer can generate one thousand lines of executable code per month that generates 10,000 logic gates. That makes 120,000 gates per year. Capacity today is, for 0.25, around 5,000,000 gates. If half is used for memory, then we need 20 engineers to fill the gap. This is without the software component, which will account for engineering resources in the same order of magnitude. Worse, if we believe Sematech year 2001, you could envision a 12 million equivalent gate SoC. Sematech states that their yardstick for production code is 100 gates per day per engineer. That's in the order of 4,000 gates per month, 50,000 gates per year. A 20-engineer design team leaves a Silicon White Space Hole of 3 million gates for additional hardware. You have no other alternative than reusing what you have already created, and therefore you have no other alternative than adopting a design for reuse strategy.

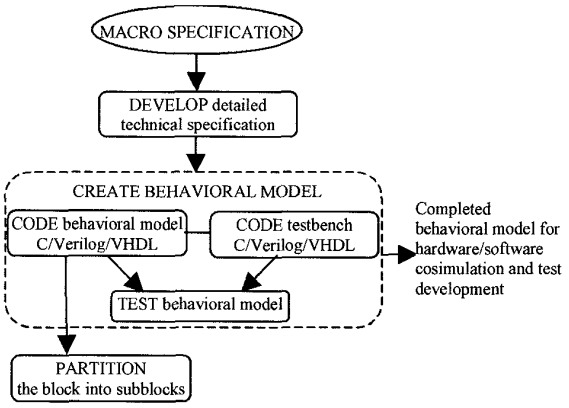


Figure 2. The Macro Design Process

How to create a Reusable Macro

We will distinguish between a soft and a hard macro. This is what is generally accepted by the industry. The firm macro representation, although well defined in the VSI Alliance standards (3) is an intermediate step between the soft and the hard macro that is usually internal to the creator. First common sense rule: "Before a macro can be reusable, it has to be usable" (4). So good design practices that have been enforced for years like good documentation, good coding style, thorough commenting, well-specified test plans, do not all of a sudden disappear. The design for reuse will add to the overall development costs the payoff being spread over many designs and again by the dominant factor that the silicon is available and the law of business pushes the design engineers to use it. The extra development time to make the core reusable will be, again, mostly the enforcement of good design practices that commercial ASIC vendors have been using for years: a macro should be as much configurable as possible, RTL code should be as generic as possible over available simulation and verification tools, synthesis scripts should be consistent over multiple libraries in the same class of semiconductor processes, testbenches should be stand-alone, compliance test suites should exist, a prototype should exist running application software, and documentation should be of very high quality. Again nothing revolutionary, maybe nothing new, you just have to enforce the rules and make it happen for the SoC.

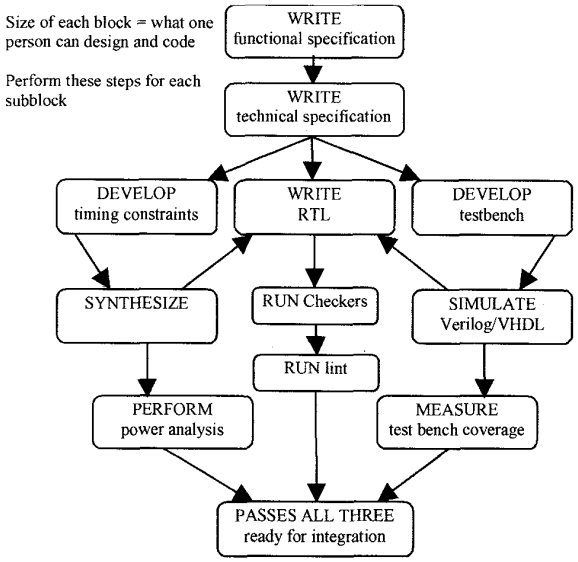


Figure 3. Sub-blocks Design Flow

The starting point for any design project consists in writing the specifications. An overview of the specification flow is given in fig. 1. An interesting debate is emerging between formal specifications and executable 'specifications. The design community writes almost exclusively executable specifications, typically written in C, C++ and/or HDL. Formal specifications are the characterization of the design, defined independently of any implementation. Once you have the formal specifications, formal methods and tools can be used to check against the specifications. A typical field where work is well advanced is the domain of property checking. An identical situation exists for documentation, as we will see later. During the specification process some key decisions

Authorized licensed use limited to: ETRI. Downloaded on October 14,2026 at 02:31:45 UTC from IEEE Xplore. Restrictions apply.

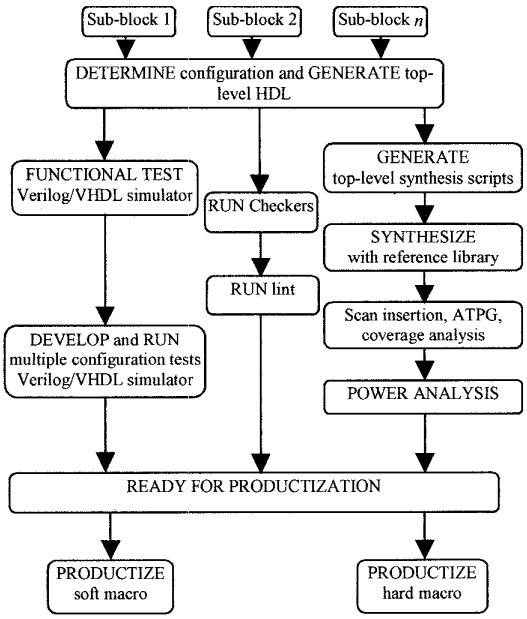


Figure 4. Sub-blocks Integration Flow

have to be made before creation or selection of the reusable IPs for your chip:

Timing and Synthesis:

- Synchronous and registered based.
- Number of clock domains and clock frequencies must be documented.
- Reset strategy for the chip must be documented. Synchronous reset is recommended.
- Synthesis strategy must take into account timing, area and power goals. Each macro must have its independent synthesis scripts.

Functional Design Issues:

- Busing scheme must be specified
- Debug features for hardware and software must be planned.

Physical Design Issues:

- Floorplanning strategy must be defined if you plan to mix hard and soft macros.
- Floorplanning must begin as early as possible in the design process. Floorplanning will be critical in the interface strategy and therefore in the integration process.
- Clock Distribution.

Verification Strategy:

We will develop this topic more in details later.

Test Strategy:

- Scan-BIST-Boundary Scan.

Once system-level specification and verification strategies are frozen, each macro to be created or integrated will be broken down from the top-level macro view, as described in fig. 2 (Fig. 4-4, RMM), into sub-blocks manageable by a small and autonomous design team. Once these sub-blocks are well specified and well documented, the teams break-off. These teams will then execute to the sub-blocks design flow, as shown in fig. 3 (Fig 4-5, RMM). The sub-block design flow introduces relatively new tools, HDL specific checkers, lint and checkers that will help stopping the exponential growth of the verification process by assisting in the creation of a robust soft macro. Where tools and new methodologies are needed is in the field of test suites generation and application testbenches generation at HDL level. Code coverage tools are available but still very rudimentary. Some extremely interesting technology is being announced that could bring some relief in this field. This new software verification technology will also have a major impact on curtailing the verification process growth. The more we verify at HDL level, the easier the final verification is. The sub-block teams get back together for the top-level integration and run final test before productization as described in fig. 4 (Fig. 4-6, RMM). This is a very important step because it will be the first grading of 'how much reusable' the sub-blocks were created. Ease of integration and speed of verification will be the measure. Productization of the soft core as shown in fig. 5 (Fig. 4-7, RMM) has some interesting steps. First, a reusable soft core has a silicon prototype and test board as deliverables. Prototypes, debug system, emulation systems are common deliverables for industry standard ICs. Extending this type of deliverable to a reusable core is

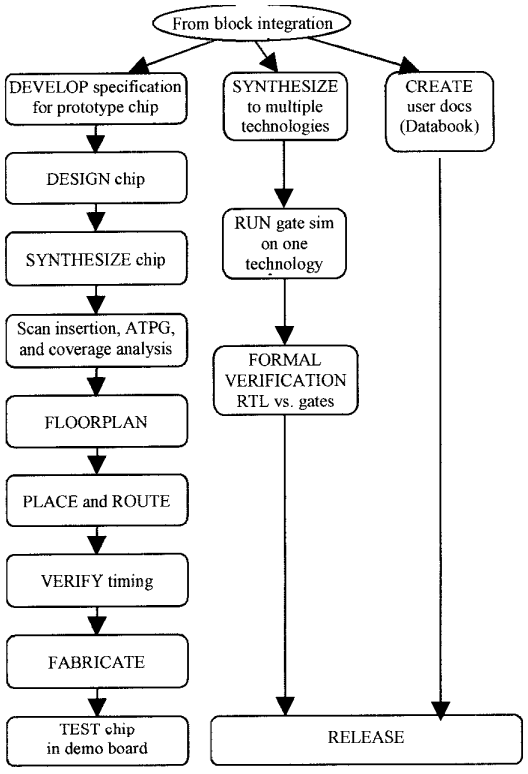


Figure 5. Core Productization Flow

Authorized licensed use limited to: ETRI. Downloaded on October 14,2026 at 02:31:45 UTC from IEEE Xplore. Restrictions apply.

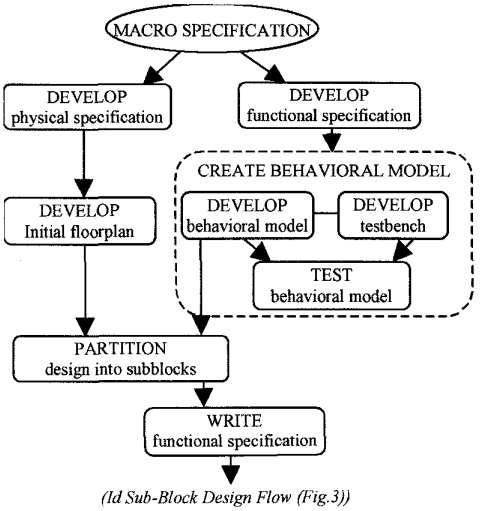


Figure 6. The Hard Macro Design

innovative. We will see the importance of this deliverable for the system-level verification. Second in the productization flow, Formal Verification is introduced. Maybe not innovative, but re-assuring that we are not on the thought process that formal verification is 'naturally' built-in in the synthesis tools.

SoC design will use hard macros. An interesting debate emerges, which is out of the scope of this tutorial, as to who owns the hard macro, due to the strong silicon technology dependency. Traditionally one would state the owner is the silicon vendor, but one could also debate that the soft core's creator is the owner.

It seems natural that the hard macro design reuse process be an extension of the soft reuse process. True and not true. The hard macro design reuse process will take into account at an early stage the physical constraints as shown in fig 6 (Fig. 8-1, RMM). This is very important to understand. The hard macro design reuse process is not the serial extension of the soft design reuse process. Then we will go through the same process as with soft macro, sub-block integration, as pictured in fig. 7 (Fig. 8-2, RMM) and productization in fig. 8 (Fig. 8-3, RMM). One key specific hard macro deliverable is a super set of soft core models for the integrator. Ideally we are talking about:

- Behavioral or ISA for simulation.
- Bus functional model for system-level verification.
- Full functional, cycle-accurate for accurate simulation.
- Functional model for emulation.
- Timing for status timing analysis.
- Floorplanning model.

Once this process of creating the reusable soft and/or hard core is done, you are left with the final process, which is to package the deliverables and make a complete product. Key deliverable for the complete package is the user guide. This is the document that will be used for the macro selection, integration and verification. The user guide is not just a merging of all documents such as specifications, synthesis scripts and whatever the authors had time to write. It is a very professional document with relevant information such as: architecture, block diagrams, I/Os, timing diagrams, exceptions to coding/design guidelines, test strategy, clocking strategy, recommended software environment including compilers and drivers, system verification strategy, debug strategy and history of the core. I stress that documentation will be a key decision maker and time saver for the SoC designer using a macro created externally and thus having no access to the IP creator.

SoC Integration with Reusable IPs

The SoC team has partitioned the chip into various blocks: for example, processors/controllers, memory/memory-controllers, I/O interfaces, Bus interfaces, data manipulation functions and software drivers-protocols. During this partition task, the selection of the reusable blocks, soft or hard, will be investigated by the SoC team. At this level of the project the most critical factors affecting the choice will be: quality of documentation, robustness of the chosen verification environment, and usability track record of the blocks. If the blocks have been created and documented for easiness to use, then the integration process is very straightforward. If the blocks are each a separate individual jewel based around performance-area-power, but are isolated islands, then integration will be a nightmare. As a summary, fig. 9 (Fig. 10-1, RMM) gives the overview of integrating reusable blocks in an SoC. Nothing innovative in the block integration flow. We didn't write much about memory or datapath structures, which are referenced on the flow diagram. These structures have historically been the antidote to

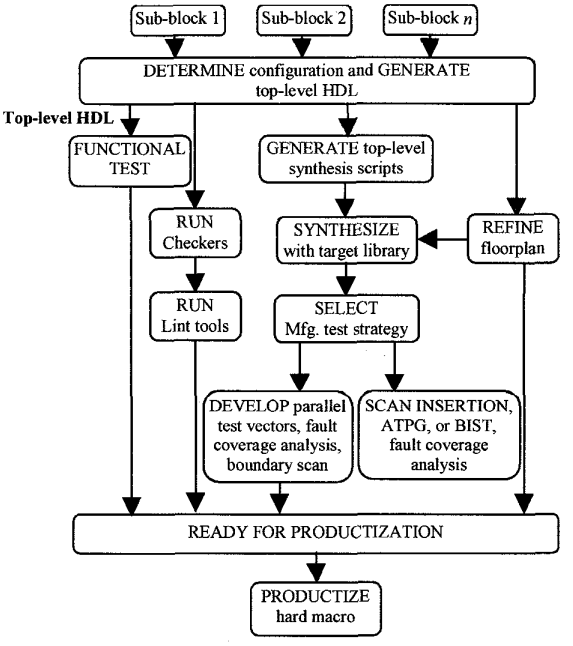


Figure 7. Integrating Sub-blocks into a Hard Macro

Authorized licensed use limited to: ETRI. Downloaded on October 14,2026 at 02:31:45 UTC from IEEE Xplore. Restrictions apply.

justifying non-reusability. Memory design is by construction almost not portable or reusable. Memories take over almost fifty percent of the overall silicon of the SoC. Therefore the objective will be the smallest, fastest and minimum power. On top of that the basic element of a memory element, the memory cell, is very technology dependent. Some progress has been made towards portability and reusability by having the user interface at the highest possible level for the configuration inputs, which is usually textual through some kind of graphical interface. On the other hand the datapath element has significantly improved in portability and reusability. Time-to-market pressure, and therefore shorter development time, have moved very quickly datapath tools and methodologies from handcrafted design style datapath to tools where you enter the structural description for the design in a flexible HDL. The crucial evolution was when datapath synthesis appeared and could generate on optimal netlist for the entire datapath. Once it was accepted that from this optimal netlist, the target goals for the key indicators, performance, productivity and portability could be as good, if not better, than with handcrafted and specialized functions, then datapath technology development made tremendous progress towards reusability. Standard place and route tools can deliver high performance datapath layouts from the optimal netlist. Therefore you can create technology independent designs without sacrifying performance.

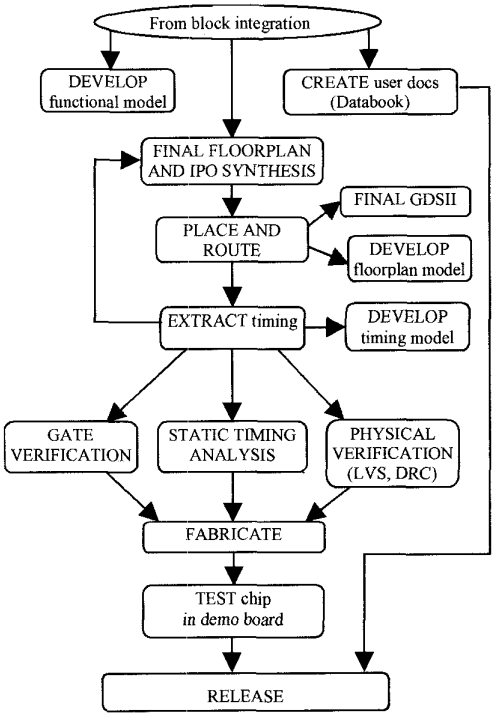


Figure 8. Hard Macro Productization

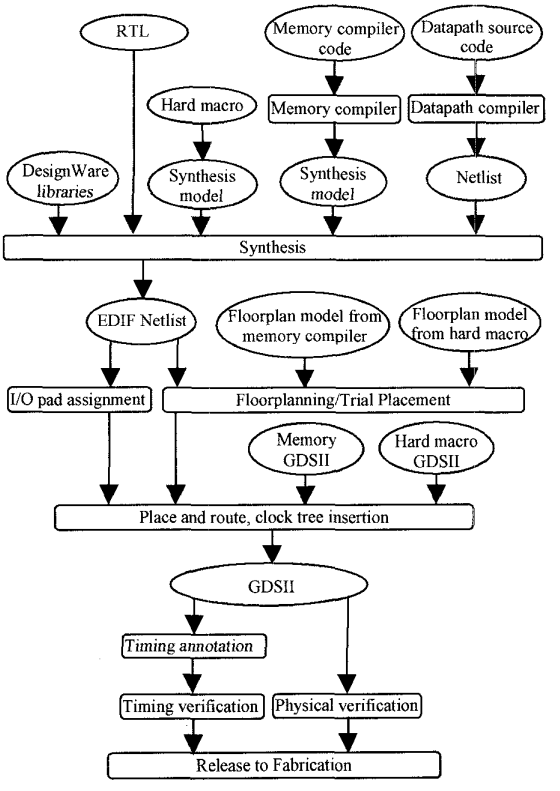


Figure 9. Integrating Blocks into a Chip

SoC Verification

The SoC or IP verification process is a high level of concern in the design and industrial community. The overall verification process which is an integral part of the SoC design process will depend essentially on the following factors: quality of the test plan models, testbenches, robustness of reusable IP blocks and the verification flow. Essential to execution of the test plan is to check that all major/embedded IP blocks have been thoroughly verified and are reusable. This will be the first big return on investment to the reusable effort. Just as an example, an embedded processor can have more than 500 test suites. The processing power to run the test suites, mostly compliance tests, can be several weeks for one core.

Therefore the verification will be limited to the interblocks which can be reduced in principle to transaction verification. Today the straightforward verification scheme is to run an RTL testbench of the top-level RTL that activates all blocks and interactions between the blocks. Observing transaction activity at primary I/O and memory contents would allow to state that the chip is working correctly. Several new trends are being investigated in transaction verification. The first one is to develop at RTL level a monitor of the process. Two types of monitors are being used: monitors coded with the RTL blocks in what is called a point-to-point process presented in fig. 10 (Fig. 11-1 (a)), or the Bus Transaction Monitor (BTM) presented in fig. 11 (Fig. 11-1 (b), RMM). This is by far the best solution for flexibility where you can create 'Per-

Authorized licensed use limited to: ETRI. Downloaded on October 14,2026 at 02:31:45 UTC from IEEE Xplore. Restrictions apply.

sonality Modules' for each IP hooked up to the common bus. The BTM allows total control on the sequence of transactions to test. The verification of the behavioral of the blocks themselves during transaction can be done with a mix of bus functional models for the blocks not under verification, and full RTL model of the block under verification. Another difficult element in the verification process will be the embedded memories. We mentioned two important design functions in the SoC integration paragraph, underlining that the datapath has almost become a potential candidate for reusability, but that memories are still a major bottleneck. For SoC verification, a great amount of efforts are being produced to tackle the embedded memory verification at behavioral level, which is an important step to reusability. Visualizing the behavior of an embedded memory as a finite state machine is a very innovation idea. This representation of embedded memories at the behavioral level and associated test guidelines is a major step in the right direction for this critical SoC function (5). Then time has come for the SoC team to face the task of verifying that the complete chip runs real applications. By understanding the magnitude of this challenge, the SoC team has two approaches. Either you increase the level of abstraction of the blocks, or, you use a specialized hardware environment for the verification. The first approach being pretty straightforward, only the second approach will be discussed in this tutorial. The specialized hardware approach is extremely efficient if during the soft and hard core productization process, silicon prototypes and all the necessary models were developed with the SoC verification strategy in mind. This will re-enforce the

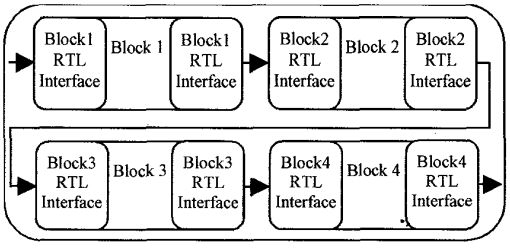


Figure 10. Chip with point-to-point Interfaces

importance of specifying the chip verification strategy as the system-level specification process. The specialized hardware is a global solution to the SoC hardware/software verification problem. Verification tool interoperability between hardware and software will be fundamental at the final verification stage. The investment is significant because the challenge is great. You cannot patch verification tools together. You need a guaranteed global solution. A solution is proposed in fig 12 (Fig. 11-5, RMM) which takes into account all aspects of the SoC verification and maximizes the effort done during the creation process of the reusable cores and integration. The kernel of the hardware verification system is the emulator. All blocks and equivalent models are mapped in the emulator system hardware. Even the tricky embedded memory blocks are emulated. The stimulus world of synthesizable testbenches and test vectors can reside on the emulator, and therefore can run at full speed. Behavioral testbenches will reside on the host. The system software/hardware environment allows you to run the software debug tools and the hardware/software cosimulation tools. Probably the key feature of the total system verification environment is the physical environment where we can plug-in physical boards with the silicon prototypes that were created during the IP creation process. Even one step further, you can add to these boards hardware testbenches such as signal generators. Last but not least, you can plug-in a complete in-circuit emulation system, and therefore debug an application board. One of the most cost-effective applications of in-circuit verification is the fact that you can test systems that run with the real asynchronous world. You can plug the in-circuit emulation in the world of random daily, weekly, monthly events, log the results and analyze the information. Capacity should not be an issue for this type of verification environment. The latest announcements claim emulation capacities of more than 25 million equivalent gates with more than 100 slots for add-on boards.

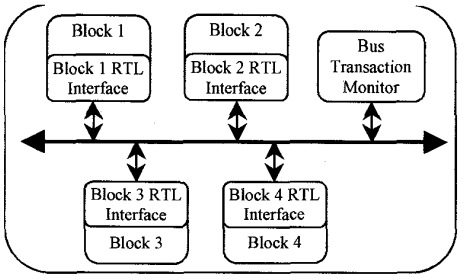


Figure 11. Chip with an on-chip Bus

Software Driven Verification

We intentionally did not discuss the software aspect of the SoC creation, integration and verification in this tutorial, as stated at the beginning of the paper but I think we need to at least set the stage for a very elegant integrated solution with the hardware specialized verification environment. Goals of software-driven verification are obviously verification of the hardware using real software, and verification of the software using an emulated hardware SoC. As stated above, the new capacities offered by the emulators allow you to test the software on the emulator running at near-real-time speeds. Development and debugging of low-level hardware device drivers can almost be done in real-time. Tremendous productivity gains can be accomplished using this type of verification environment.

Documentation

Impossible not to stress again the importance of documentation in general, and particularly for reusable IPs. A SoC design will be in excess of 10 million gates. Can we imagine that one or maybe two (super) engineers can integrate all the components of the SoC? Could we imagine that the Intel or IBM processors do not have a thorough documentation? No, we cannot. But we can imagine that

Authorized licensed use limited to: ETRI. Downloaded on October 14,2026 at 02:31:45 UTC from IEEE Xplore. Restrictions apply.

we will accept to integrate macros without the proper documentation. Many decisions will be taken during the SoC design process based on the documentation. The accuracy of the information is as important as the quality of the HDL code. But we do not have automatic documentation generators, or lint and coverage equivalent tools. The reusable IP is high quality and all information to select the right IP, integrate it in your SoC and verify that the SoC works, hardware and software, should be documented. Most IP providers supply a vague product specification that has everything mixed into one document: architecture and functional description, detailed description of I/O, block diagram, design flow, tool dependent restrictions, testbenches data, installation data, you name it. Some have separate installation guides, README files, application notes. The content of the User Guide or 'Why and How' to use the core is common sense. Just take the time to discuss with a senior SoC designer and ask him to go through his thought process on selecting an IP, integrating the IP and verifying the IP and you will have it defined. Why does this task of the Reusable IP design process get the lowest attention? Should we invest more in the automatic documentation generation process? I think that if we give more thought to the issue, as to its implementation, we will come to the same conclusion as a couple of years ago for verification: we now have verification teams. Therefore we should not ask the design or integration teams to write the user guide. This should be a well-defined task in the overall project and the responsibility of a well-defined team.

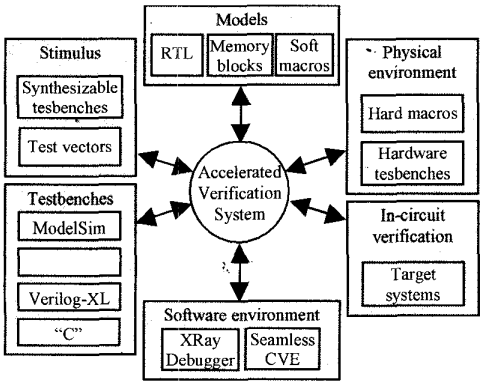


Figure 12. Mentor Graphics Accelerated Verification System Process

Conclusion

System-on-a-chip is the paramount challenge of the electronic industry for the next millennium. The semiconductor industry has delivered what we were expecting and was predicted: silicon availability for over 10 million gates. VSI Alliance and RAPID organization have defined industry standards and data formats for SoC. The first 'How-to-Do' book to create reusable IPs for System-on-a-Chip designs has been published. EDA tool providers understand the issues and are proposing new tools and solutions on a quarterly basis. If the beginning of the millennium for the electronic chip industry is the SoC era, then reusability must be the engine.

Acknowledgements

I would like to thank Nathalie Rocher for all the effort and talent put in making this document available on time. I would also like to thank Thierry Sarrazin and Christophe Dejean, from the Inventra IP Factory, for their valuable inputs. Also special thanks to Carl Harris, from Kluwer Publishers, for his support and kind authorization to publish flow diagrams extracted from the Reuse Methodology Manual.

References

(1) Michael Keating and Pierre Bricaud, "Reuse Methodology Manual for System-on-a-Chip Designs", Kluwer Academic Publishers, 1998.

(2) B, Braune, Senior VP Mentor Graphics "The Year 2000 Challenge.

(3) VSI™ Alliance, "VSI Architecture Document", Version *1.0*.

(4) Richard Goering, "A series on the Reuse Methodology Manual", 5 articles published in EE Times.

(5) Tony Tsang, Ching Li, Mozammel Hosain, Mentor Graphics Corporation, "A practical verification strategy for ASIC embedded memory behavioral models". IP98 Conference proceedings.

Authorized licensed use limited to: ETRI. Downloaded on October 14,2026 at 02:31:45 UTC from IEEE Xplore. Restrictions apply.